Homework 6: The Prerequisite Planner

CS 311: Data Structures & Algorithms (Fall 2026)

Points: 100. **Due:** Wednesday, November 18, 2026 at 11:59 PM Central.

Starter repository: GitHub template. Python 3.11+; standard library only.

The Mission

Lab 12's **USI Campus Connector** used BFS to find a route with the fewest building-to-building links. Now use **DFS** to produce a valid course registration order and detect prerequisite cycles. An arrow points from a prerequisite to the course that depends on it. Use the supplied course graph as a teaching model, not an official USI degree audit.

Part A: DFS Timestamps & Ordering (50 Points)

Instructions: Answer Questions 1–4 in a separate PDF or Markdown document, using matching numbers. Review slides 54–62 and the worked trace on slide 61.

1. **Why DFS, Not BFS, For This Job:**

A **topological order** places each vertex before every destination of its outgoing edges. A DFS-based algorithm constructs such an order on a DAG using each vertex's **finish time**, when its recursive call returns. Explain in one or two sentences why BFS, which processes nodes level-by-level rather than one deep path at a time, doesn't naturally produce finish times the same way DFS does.

2. **Visual Tracing — DFS Discovery/Finish Times:**

Given the prerequisite graph with directed edges `CS115 -> CS215`, `CS115 -> CS258`, `CS215 -> CS311`, `CS258 -> CS311`, `CS311 -> CS411` (an edge `A -> B` means "A is a prerequisite for B"), trace a DFS starting from `CS115` (visiting outgoing edges in alphabetical order of the destination course). Record the **discovery time** and **finish time** of every vertex, using a single incrementing clock across the whole traversal.

3. **From Finish Times to Topological Order:**

State the rule that converts a set of DFS finish times into a valid topological order, and apply it to your Question 2 trace to produce the final course registration order.

4. **Cycle Detection via Edge Classification:**

During a DFS, an edge from the current vertex to an **ancestor still on the current recursion stack** is called a "back edge," and its presence means the graph has a cycle. Explain how you can detect a back edge during the traversal using only the information you already have at each recursive call (hint: track which vertices are on the current path, not just which have been visited overall) — and explain why a plain "already visited" check by itself is not enough to distinguish a back edge from a perfectly normal DAG edge that happens to point to an earlier-visited vertex.

Part B: The Scheduler (50 Points)

Instructions: Complete has_cycle and topological_sort in prereq_planner.py. Clone your own template copy, open its folder in VS Code and run python test_planner.py. The untouched starter stops at its first NotImplementedError; that is the intended TODO.

Requirements:

1. `has_cycle(graph: Dict[str, List[str]]) -> bool` must be implemented using **recursive** DFS, tracking both a permanent "visited" set and a temporary "on the current recursion path" set (per Part A, Question 4) to correctly detect back edges. Must work correctly on a graph with multiple disconnected components.
2. `topological_sort(graph: Dict[str, List[str]]) -> Optional[List[str]]` must return a valid topological order as a list, or None if the graph contains a cycle (checked via `has_cycle` first). Must be implemented using DFS finish-time ordering (per Part A, Question 3), not Kahn's algorithm (in-degree counting) — that's a different, equally valid technique, but this assignment is about connecting DFS timestamps to a real use.
3. Both functions must correctly handle a graph with an isolated course (no prerequisites, nothing depends on it) and an empty graph.
4. **Base Case Required:** The recursive DFS helper(s) must correctly terminate when a vertex has no unvisited neighbors left. Remember the **Brutality Clause** — a recursive method with no base case is an automatic 0.
5. **Verification:** Run the script using `python test_planner.py`. It replays the Part A, Question 2/3 graph and checks your topological order against every constraint (every prerequisite appears before what depends on it), confirms `has_cycle` correctly flags a graph you know contains a cycle, and stress-tests `topological_sort` on random DAGs by verifying the constraint-satisfaction property rather than an exact ordering (multiple valid topological orders can exist for the same graph). If the supplied checks pass, the script prints the **Success Token**. This token covers those checks; recursive-method compliance, written reasoning, and behavior on other inputs still require review.

Submission

1. `HW6_Theory.pdf` (or `.md`)
2. `prereq_planner.py` (your completed code)
3. The **Success Token** generated by `test_planner.py`

Submit through **Blackboard** and include the syllabus **AI Usage note** when AI tools were used. Part A remains 50 points and Part B remains 50 points. The recommended work order and slide references add no new submission item.